// BOOTCAMP · 2026 · DAY 1 OF 2

# GitHub Copilot + Actions

Day 1: GitHub Fundamentals and Copilot Configuration

10:00 AM – 5:00 PM



- Git & GitHub Fundamentals
- Repository Management
- Branching & Pull Requests
- Copilot Configuration in VS Code
- AI Code Generation
- Debugging with Copilot
- Prompt Engineering

AUDIENCE: Developers & GitHub Admins

FORMAT: Lecture + Hands-On Labs

LABS: 3 Hands-On Exercises

// DAY 1 AGENDA

# What We're Building Today

10:00 AM – 12:00 PM

## Block 1: Git & GitHub Fundamentals

- Introduction to Version Control
- Core Git Commands
- Creating & Managing Repositories
- Branching & Merging Strategies
- Collaborating with Pull Requests

**LAB 1: Repo Setup & PRs**

1:00 PM – 3:00 PM

## Block 2: GitHub Copilot Configuration

- Introduction to GitHub Copilot
- Installing Copilot in VS Code
- First Inline Suggestions
- Copilot Chat Interface
- Debugging with AI Assistance

**LAB 2: Install & Configure**

3:15 PM – 5:00 PM

## Block 3: Leveraging Copilot's Features

- Code Generation Capabilities
- Multi-Language Code Snippets
- Writing Functions with Copilot
- Prompt Engineering Techniques
- Security & Responsible Use

**LAB 3: Build a REST API**

LUNCH: 12:00 – 1:00 PM

BREAKS: Mid-morning & Mid-afternoon

LABS: 3 hands-on exercises

// LEARNING OBJECTIVES

# By End of Day, You Will Be Able To...

Six concrete skills you will practice and demonstrate through hands-on labs today.

- Developers
- GitHub Admins
- Team Leads

## Day 1 Learning Objectives

**01 Version Control Fundamentals**

Explain the role of Git and GitHub in modern software development teams

**02 GitHub Collaboration**

Create and manage repositories, branches, and pull requests effectively

**03 Copilot Setup**

Install and configure GitHub Copilot in VS Code with custom project instructions

**04 AI-Assisted Coding**

Use inline suggestions and Copilot Chat to generate working, quality code

**05 AI Debugging**

Debug code and identify issues using Copilot's slash commands (/fix, /explain)

**06 Prompt Engineering**

Fine-tune Copilot output using comment-driven development and context techniques

// SECTION 1: GIT & GITHUB FUNDAMENTALS

# Version Control Is the Foundation of Every Modern Software Team

### Without Version Control

- Overwritten files and lost work when collaborating
- "final_v3_REAL.zip" file naming chaos
- No audit trail of who changed what, or why
- Fear of breaking working code when experimenting

### With Version Control

- Full, permanent history of every change ever made
- Parallel work streams without interfering with others
- Safe experimentation with the ability to rollback instantly
- Clear accountability and documentation of intent



### Git

The world's most widely used distributed version control system. It tracks changes locally and manages the history of your source code.



### GitHub

The cloud platform built on top of Git. It adds team collaboration, access control, CI/CD automation, and project management.

**100M+** DEVELOPERS USE GITHUB AS OF 2024

// GIT FUNDAMENTALS

# Git Tracks Changes Across Three Stages




**Note:** The staging area acts as a "loading dock" where you decide exactly what goes into each shipment (commit) before it leaves.

// CORE GIT COMMANDS REFERENCE

# The Commands You Will Use Every Single Day

| COMMAND | WHAT IT DOES | WHEN TO USE IT |
|---|---|---|
| `git clone <url>` | Copy a remote repo to your machine | Starting work on an existing project |
| `git status` | Show changed/staged files | Before every commit |
| `git add <file>` | Stage a file for commit | After editing files |
| `git commit -m "msg"` | Save a snapshot with a message | After staging changes |
| `git push` | Upload commits to GitHub | After committing locally |
| `git pull` | Download and merge remote changes | Before starting new work |
| `git log --oneline` | View commit history | Reviewing what changed |

**PRO TIP:** These 7 commands cover 90% of daily Git usage.

// REPOSITORY MANAGEMENT

# A Repository Is Your Project's Home on GitHub

A repository contains all of your project's files and each file's revision history. You can discuss and manage your project's work within the repository.

### Public Repos

Visible to everyone on the internet. Ideal for open source projects and portfolios.

### Private Repos

Visible only to invited collaborators. The standard choice for proprietary company code.

### Key Settings

Configure branch protection rules, manage collaborator access, and set topics/descriptions.

### README.md

The front door of your repository. Always include one to explain what the project does.

### .gitignore & LICENSE

**.gitignore** prevents secrets, build artifacts, and OS files from being committed.
**LICENSE** defines how others can use, modify, and distribute your code.

```
# Example .gitignore
node_modules/
.env
dist/
.DS_Store
```

// BRANCHING STRATEGIES

# Branches Let Teams Work Simultaneously Without Breaking Each Other's Code

## BRANCH TYPES

### Main / Master Branch

Production-ready code — always deployable.

### Feature Branches

Isolated work on a single feature or fix.

**RULE OF THUMB**

**One branch per task. Keep branches short-lived (< 1 week).**

## COMMON STRATEGIES

### GitFlow

`main + develop + feature/release/hotfix branches`

- Suited for scheduled releases

### GitHub Flow

`main + feature branches + PRs`

- Suited for continuous delivery

### Trunk-Based Development

`Everyone commits to main frequently with feature flags`

- Suited for high-velocity teams

// BRANCHING COMMANDS IN PRACTICE

# Creating, Switching, and Merging Branches

| COMMAND | PURPOSE |
| --- | --- |
| `git branch feature/user-auth` | Create a new branch |
| `git checkout feature/user-auth` | Switch to the branch |
| `git checkout -b feature/user-auth` | Create AND switch (shortcut) |
| `git merge feature/user-auth` | Merge branch into current branch |
| `git branch -d feature/user-auth` | Delete a branch after merging |
| `git branch -a` | List all branches (local + remote) |

### MODERN GIT ALTERNATIVE

`git switch` and `git restore` as clearer alternatives to `git checkout`. Both work perfectly, but `checkout` remains the most common in documentation and tutorials.

### MERGE CONFLICTS

When merging, if the same file was changed in both branches, Git will pause and ask you to resolve the conflict manually before completing the merge. This is a normal part of team collaboration.

// COLLABORATION WORKFLOW

# Pull Requests Are Where Code Gets Reviewed, Discussed, and Approved

A Pull Request (PR) proposes merging one branch into another, providing a structured space for code review, discussion, and approval.

## ANATOMY OF A GOOD PR

- **Clear Title:** Summarizes the change concisely.
- **Detailed Description:** Explains what changed, why it changed, and how to test it.
- **Linked Issue:** Connects the code change to the original task or bug report.

**GitHub Copilot Integration:** Copilot can automatically generate comprehensive PR summaries based on the commit history and code diffs, saving developers time.

## REVIEWER ACTIONS

### Approve

Sign off on the changes, allowing the PR to be merged.

### Request Changes

Block merging until specific feedback is addressed.

### Comment

Leave general feedback or inline comments on specific lines.

### Suggest Edits

Propose exact code changes that the author can commit directly.

## MERGE STRATEGIES

- Create a merge
- Squash and merg
- Rebase and merge

// REAL-WORLD PR WORKFLOW

# The Professional Pull Request Lifecycle

01 **CREATE BRANCH**
Create a feature branch from `main` to isolate your work.

02 **MAKE COMMITS**
Make commits with clear, atomic messages describing the changes.

03 **PUSH TO GITHUB**
Push your local feature branch to the remote GitHub repository.

04 **OPEN PULL REQUEST**
Open a PR with a detailed description and link to the relevant issue.

05 **CI/CD CHECKS**
Automated tests and checks run automatically (Day 2 topic).

06 **PEER REVIEW**
Peer reviewer approves or requests changes to the code.

07 **ADDRESS FEEDBACK**
Address reviewer feedback with additional commits on the same branch.

08 **MERGE PR**
Merge the branch into `main` after approval and passing checks.

09 **CLEANUP**
Delete the feature branch to keep the repository clean.

10 **DEPLOY**
Deployment triggers automatically based on the merge to main.

// ADVANCED GIT

# Advanced Git Workflows: Git Flow vs GitHub Flow

## GIT FLOW

- **Strict Branch Hierarchy:** Uses main, develop, feature, release, and hotfix branches.
- **Scheduled Releases:** Designed for software with explicit version numbers and release cycles (e.g., v1.2.0).
- **High Overhead:** Requires significant discipline to maintain multiple long-running branches.

BEST USED FOR:

*Desktop applications, mobile apps, or enterprise software with strict compliance and manual release gates.*

## GITHUB FLOW

- **Simplicity:** Only one long-running branch (`main`). All work happens on short-lived feature branches.
- **Continuous Delivery:** The `main` branch is always deployable. Merging a PR often triggers an automatic deployment.
- **Low Overhead:** Focuses on speed, peer review (PRs), and automated testing rather than complex branch management.

BEST USED FOR:

*Web applications, SaaS platforms, microservices, and teams practicing CI/CD and Agile methodologies.*

// ADVANCED GIT

# Handling Merge Conflicts with Confidence

### What is a Conflict?

A conflict occurs when Git cannot automatically merge branches because two people modified the **exact same line** of a file, or one person edited a file while another deleted it.

### Don't Panic

A conflict is not an error; it is Git pausing to ask for human intervention. Your code is not broken or lost. Git has simply injected markers into the file to show you the competing changes.

### The Resolution Process

1. Open the conflicted file in VS Code.
2. Look for the conflict markers.
3. Choose which code to keep (or combine them manually).
4. Delete the markers, save, and commit.

server.js

```
const express = require('express');
const app = express();

<<<<<<< HEAD (Current Change)
const PORT = process.env.PORT || 3000;
console.log(`Starting server on port ${PORT}`);
=======
const PORT = 8080;
console.log(`Listening on ${PORT}`);
>>>>>>> feature/update-port (Incoming Change)

app.listen(PORT);
```

// ADVANCED GIT

# Git Stash: Saving Unfinished Work Safely

The scenario: You are halfway through writing a new feature when a critical bug is reported. You need to switch branches immediately to fix it, but your current code is broken and **not ready to commit**. Git won't let you switch branches with a dirty working directory. The solution is the Stash.



### Working Directory

Contains your messy, half-finished code that breaks the build.

`git stash` →

← `git stash pop`



### The Stash (Clipboard)

A temporary storage area where Git shelves your changes, leaving your working directory clean.

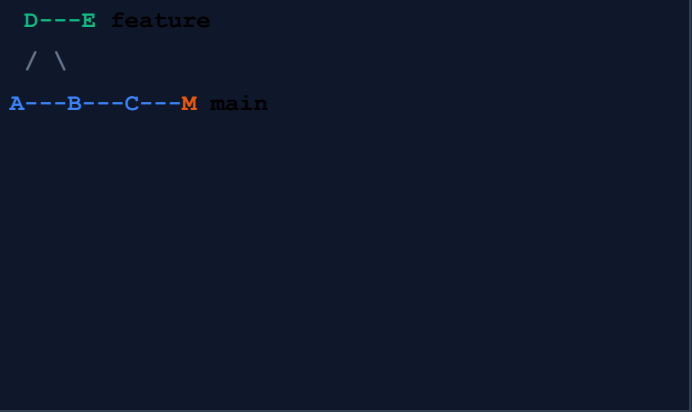
| | |
|---|---|
| `git stash` | Takes your modified tracked files and saves them on a stack of unfinished changes. |
| `git stash list` | Shows all the stashed changes you currently have saved. |
| `git stash pop` | Removes the most recent stash from the stack and applies it back to your working directory. |
| `git stash drop` | Permanently deletes the most recent stash without applying it. |

// ADVANCED GIT

# Rewriting History: Rebase vs Merge

## Git Merge

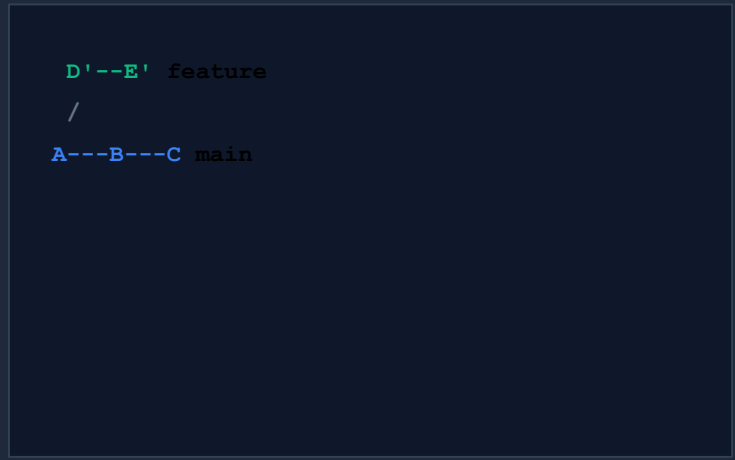
Combines two branches by creating a new "merge commit". Preserves the exact history of when and where changes happened.




**Pros:** Non-destructive, true historical record.
**Cons:** History can become a messy, tangled web of merge commits on large teams.

## Git Rebase

Moves the base of your feature branch to the tip of main. It literally rewrites history to make it look like you started working today.




**THE GOLDEN RULE OF REBASE**

Never rebase commits that have already been pushed to a public repository that others are using. It will destroy their local history.

// ADVANCED GIT

# Git Log & Blame: The Developer's Audit Trail

## The Project History

```
$ git log --oneline --graph
 * a1b2c3d (HEAD -> main) Merge pull request #42 <alice>
|\
| * f9e8d7c Fix off-by-one error in loop <bob>
| * b4a5c6d Add retry logic to API call <bob>
|/
* 9f8e7d6 Update dependencies to v2.0 <charlie>
* 1a2b3c4 Initial commit <alice>

$
```

## Line-by-Line Investigation

api.js

```
12  async function fetchData() {        Alice, 2 years ago • Initial commit
13      let retries = 3;                Bob, 2 days ago • Add retry logic
14      while (retries > 0) {           Bob, 1 day ago • Fix off-by-one
15          try {                       Alice, 2 years ago • Initial commit
16              return await            Charlie, 1 month ago • Update deps
                axios.get(url);
```

// SECTION 2: COPILOT CONFIGURATION

# You Know How to Collaborate — Now Let's Accelerate

**55%**

Studies show Copilot users complete tasks up to 55% faster than developers without AI assistance.

### Foundation Established

You now have the Git and GitHub foundation every developer needs to collaborate effectively.

### AI Pair Programmer

GitHub Copilot is an AI pair programmer built directly into your editor, working alongside you.

### Massive Training Data

Copilot is trained on billions of lines of public code and powered by advanced large language models.

### Contextual Understanding

It does not just autocomplete — it understands context, intent, and patterns across your entire project.

// WHAT IS GITHUB COPILOT?

# GitHub Copilot Is Your AI Pair Programmer

### ORIGINS

Launched in 2021, built through a partnership between GitHub and OpenAI.

### AI MODELS

Powered by advanced large language models (currently GPT-4o, Claude, Gemini, and more).

### AVAILABILITY

Available across multiple tiers: Copilot Free, Pro, Pro+, Max, Business, and Enterprise.

### ECOSYSTEM

Works seamlessly in VS Code, Visual Studio, JetBrains IDEs, Neovim, GitHub.com, and the CLI.

## Inline Suggestions

**Code as you type.** Copilot anticipates your intent and suggests entire lines or functions in ghost text. Press Tab to accept.

## Copilot Chat

**Conversational AI.** A dedicated chat panel to ask questions, request code generation, explain errors, and generate tests.

### DEEP CONTEXTUAL AWARENESS

**Copilot understands your entire codebase context, not just the current file. It analyzes open tabs, imports, and project structure to provide highly relevant suggestions.**

// COPILOT EDITIONS

# Choosing the Right Copilot Plan for Your Team

| PLAN | BEST FOR | KEY FEATURES |
| --- | --- | --- |
| **Free**<br>$0 / month | Individual exploration | ✓ 2,000 inline suggestions per month<br>✓ 50 chat messages per month |
| **Pro**<br>$10 / month | Individual developers | ✓ Unlimited inline suggestions<br>✓ Access to premium models |
| **Pro+**<br>$19 / month | Power users | ✓ Higher AI credit allowance<br>✓ Access to all available models |
| **Business**<br>$19 / user / month | Teams & Organizations | ✓ Policy controls & audit logs<br>✓ Organization management & IP indemnification |
| | | ✓ |

// INSTALLATION GUIDE

# Getting Copilot Running Takes Less Than 5 Minutes

STEP 01

## Open Extensions

Open VS Code and navigate to the Extensions panel using `Ctrl+Shift+X` (Windows/Linux) or `Cmd+Shift+X` (Mac).

STEP 02

## Install Copilot

Search for "GitHub Copilot" in the marketplace. Locate the official extension (publisher: GitHub) and click **Install**.

STEP 03

## Install Chat

Search for and install the "GitHub Copilot Chat" extension to enable the conversational AI interface.

STEP 04

## Initiate Sign-In

Click the new Copilot icon located in the bottom right status bar of VS Code, then select **Sign in with GitHub**.

STEP 05

## Authorize Browser

Follow the prompts to open your web browser. Authorize VS Code to access your GitHub account, then return to the editor.

STEP 06

## Ready to Code

The Copilot icon in the status bar will turn active (no slash through it). You are now ready to use AI assistance!

### VERIFICATION & TROUBLESHOOTING

**Status Icon:** A filled icon means active; a crossed-out icon means disabled globally or for the current language.

**Corporate Proxies:** If you are behind a corporate firewall and experience connection errors, you may need your IT team to allowlist copilot-proxy.githubusercontent.com and api.github.com .

// COPILOT'S TWO PRIMARY INTERFACES

# Two Ways to Work with Copilot — Inline and Chat

## Inline Suggestions

### Ghost text appears as you type

- **Triggered automatically** by context as you write code or comments.
- **Multiple alternatives:** Copilot often generates several options for the same prompt.
- **Universal support:** Works in any file type including JS, Python, SQL, Markdown, and YAML.

KEY SHORTCUTS

| | |
|---|---|
| `Tab` | Accept the full suggestion |
| `Esc` | Dismiss the suggestion |
| `Alt + ] / [` | Cycle through alternative suggestions |

## Copilot Chat

### Conversational AI panel

- **Interactive dialogue:** Ask questions, request code, explain errors, and generate tests.
- **Context-aware:** Automatically references your open files, workspace, and terminal output.
- **Slash commands:** Built-in commands like `/fix`, `/explain`, `/tests`, and `/doc`.

KEY SHORTCUTS

| | |
|---|---|
| `Ctrl+Alt+I` | Open Chat panel (Windows/Linux) |
| `Cmd+Ctrl+I` | Open Chat panel (Mac) |
| `Ctrl+I` | Open Inline Chat (modify selected code) |

// CONTEXT WINDOW

# Copilot Sees More Than Just Your Cursor Position

## WHAT COPILOT ANALYZES

- Current File
- Open Tabs
- File Name
- Language
- Imports
- Comments

### Comments as Instructions

Write what you want as a comment, and Copilot will generate the code below it. This is the most effective way to guide inline suggestions.

```
// Fetch user data from the API and handle errors
```

## CHAT CONTEXT VARIABLES

| | |
|---|---|
| `@workspace` | References your entire project in Chat. Useful for broad questions like "Where is authentication handled?" |
| `#file` | References a specific file in Chat. Example: "Explain the middleware in #file:routes.js" |
| `#selection` | References highlighted code in Chat. Automatically included when you select code and open Chat. |

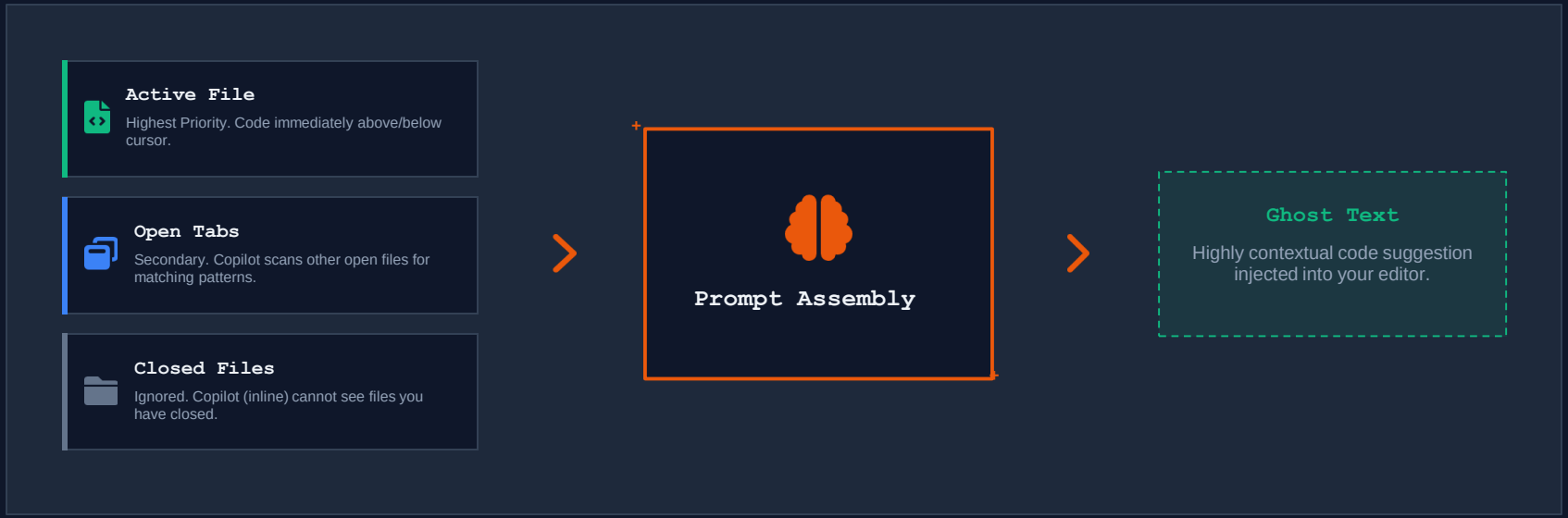
THE GOLDEN RULE

**The more context you provide, the better the suggestions.**

// COPILOT MASTERY

# Copilot Context Window: What Does It Actually See?

Copilot does not read your entire repository at once. It builds a **prompt** behind the scenes using a specific hierarchy of information to fit within the LLM's token limit. Understanding this hierarchy is the key to getting accurate suggestions.



// COPILOT MASTERY

# Managing Context: The "Open Tabs" Strategy

## BEST PRACTICES

- **Open Related Files:** If you are writing a React component, keep the CSS file, the API service file, and the data models open in other tabs.
- **Open Examples:** If you want Copilot to follow a specific pattern (like how your team writes unit tests), open a file that already uses that pattern.
- **Use Comments for Intent:** Write comments at the top of your active file explaining what you are about to do before you start coding.

## ANTI-PATTERNS

- **Hoarding Tabs:** Having 30 unrelated files open confuses the LLM. It dilutes the context window with irrelevant tokens.
- **Leaving Bad Code Open:** If you have a file open containing an insecure query or a deprecated library, Copilot will suggest code matching that bad style.
- **Assuming Global Knowledge:** Don't expect Copilot to know about a database schema just because it exists in the repo. You must open the schema file.

// COPILOT WORKFLOW

# Mastering the Copilot Suggestion Workflow

## BASIC ACTIONS

| | |
|---|---|
| `Tab` | Accept the full suggestion |
| `Ctrl` + `→` | Accept word by word (partial accept) |
| `Esc` | Dismiss the suggestion |

## CYCLING & ALTERNATIVES

| | |
|---|---|
| `Alt` + `]` Win<br>`⌥` + `]` Mac | Next suggestion |
| `Alt` + `[` Win<br>`⌥` + `[` Mac | Previous suggestion |
| `Ctrl` + `Enter` | Open suggestions panel (see up to 10 alternatives) |

### Best Practice

Do **NOT** just Tab-accept everything blindly. Read the suggestion, understand it, then accept or modify. Partial acceptance ( `Ctrl + →` ) is underused but powerful — accept just the part you want and let Copilot continue from there.

// COPILOT CONFIGURATION

# Customize Copilot to Match Your Team's Standards

## ESSENTIAL VS CODE SETTINGS

**github.copilot.enable**

Toggle Copilot on or off for specific languages. Useful for disabling suggestions in plain text files like Markdown or sensitive configuration files.

**editor.inlineSuggest.enabled**

The master switch for ghost text. If you find inline suggestions distracting during certain tasks, you can toggle this off and rely solely on Copilot Chat.

**github.copilot.editor.enableAutoCompletions**

Controls whether Copilot automatically fetches suggestions as you type, or if you must manually trigger them using a keyboard shortcut.

## ENFORCING TEAM STANDARDS

settings.json

```
{
"github.copilot.chat.customInstructions": [
{
"text": "Always use React functional components with hooks.
Do not use class components."
},
{
"file": "docs/team-style-guide.md"
}
]
}
```

**Custom Instructions** allow you to inject specific rules into every Copilot Chat prompt automatically. By committing this to your repository's `.vscode/settings.json`, you ensure every developer on the team receives suggestions aligned with your architecture and style guides.

// COPILOT CHAT

# Copilot Slash Commands Transform Debugging

## HOW TO USE COMMANDS

1. Highlight the specific block of code you want to target.
2. Press `Ctrl+I` to open Inline Chat, or open the Chat panel.
3. Type `/` to see available commands, select one, and press Enter.

## THE ULTIMATE DEBUGGING FLOW

When you encounter an error, don't just search Stack Overflow. Highlight the failing code, press `Ctrl+I`, and type `/fix` followed by the exact error message from your terminal. Copilot will analyze the code and the error together.

## /explain

Breaks down complex logic, regex, or unfamiliar syntax into plain English. Perfect for reading legacy code.

```
Example: /explain how this regex works
```

## /fix

Analyzes the selected code for bugs and proposes a solution with a diff view you can accept or reject.

```
Example: /fix TypeError: undefined
```

## /tests

Generates unit test boilerplate for the selected function, covering standard cases and edge cases.

```
Example: /tests using Jest framework
```

## /doc

Adds standard documentation comments (like JSDoc or Docstrings) to functions, classes, and methods.

```
Example: /doc in JSDoc format
```

// COPILOT MASTERY

# Copilot Chat: The /explain Command

## Step 1: Highlight Confusing Code

legacy_utils.js

```
// Inherited from a developer who left 3 years ago
export const processData = (d) => {
  return d.filter(x => x.v > 0)
  .map(x => ({...x, v: x.v * 1.2}))
  .reduce((a, c) => a + c.v, 0);
};
```

## Step 2: Ask Copilot

GitHub Copilot

`/explain` What is this highlighted code doing?

This code processes an array of objects (`d`) in three steps:

1. **Filter:** Removes any objects where the value (`v`) is 0 or negative.
2. **Map:** Creates a new array, increasing the `v` property of each remaining object by 20% (multiplying by 1.2).
3. **Reduce:** Sums up all the updated `v` values into a single total number, starting from 0.

// COPILOT MASTERY

# Copilot Chat: The /tests Command

## Step 1: Highlight Function

auth.js

```
function validatePassword(pwd) {
if (!pwd || pwd.length < 8) return false;
const hasUpper = /[A-Z]/.test(pwd);
const hasNumber = /[0-9]/.test(pwd);
return hasUpper && hasNumber;
}
```

## Step 2: Generate Coverage

GitHub Copilot

/tests Generate Jest unit tests for this function. Include edge cases.

Here are the Jest tests covering standard and edge cases:

→ Insert

```
describe ('validatePassword' , () => {
it ('returns false for short passwords' , () => {
expect (validatePassword( 'Ab1' )).toBe(false);
});
it ('returns true for valid passwords' , () => {
expect (validatePassword( 'SecurePass123' )).toBe(true);
});
});
```

// COPILOT MASTERY

# Copilot Chat: The /fix Command

## Step 1: Highlight Buggy Code

math.js

```
function findMax(arr) {
let max = 0; // Fails on negative arrays
for (let i = 1; i <= arr.length; i++) { // Off-by-one
if (arr[i] > max) {
max = arr[i];
}
}
return max;
}
```

## Step 2: Let AI Debug It

GitHub Copilot

`/fix` This function returns wrong answers for negative numbers and sometimes throws undefined errors.

I found two bugs. `max` should be initialized to the first element (not 0), and the loop condition should be `<` instead of `<=`.

```
- let max = 0;
+ let max = arr[0];
- for (let i = 1; i <= arr.length; i++) {
+ for (let i = 1; i < arr.length; i++) {
```

// SECTION 3: LEVERAGING COPILOT

# Copilot Is Running — Now Let's Use It Like a Pro

**75%**

Developers who master Copilot's advanced features report a 75% increase in coding satisfaction and focus.

### Code Generation

Understand how Copilot predicts tokens to generate accurate code snippets and entire functions.

### Comment-Driven Development

Master the art of writing clear, descriptive comments to guide Copilot's output effectively.

### Multi-Language Mastery

Leverage Copilot across your entire technology stack, from frontend frameworks to backend APIs.

### Security & Responsibility

Learn the best practices for reviewing AI-generated code and maintaining enterprise security standards.

// HOW COPILOT THINKS

# Copilot Generates Code by Predicting the Most Likely Next Tokens

## THE LLM ENGINE

Copilot does **not** search a database of code snippets or copy/paste from existing repositories. It is powered by a Large Language Model (LLM).

It reads your current context, breaks it down into chunks called **tokens** (words, parts of words, or characters), and calculates the mathematical probability of which token should logically come next based on its training data.

## IMPLICATIONS FOR DEVELOPERS

**Synthesis, Not Search**

The code generated is often entirely new, synthesized on the fly to match your specific variables and logic.

**Hallucinations Can Occur**

Because it predicts based on patterns, it can confidently suggest methods or libraries that look correct but do not actually exist.

**Human in the Loop**

Copilot is a co-pilot, not an autopilot. You must review, test, and validate its suggestions before accepting them.

### VISUALIZING TOKEN PREDICTION




* Copilot continuously recalculates these probabilities with every single keystroke.

// POLYGLOT CAPABILITIES

# Copilot Works Across Your Entire Technology Stack

## LANGUAGE SUPPORT TIERS

### Tier 1: Exceptional

Trained on massive datasets. Highly accurate and idiomatic.

Python | JavaScript | TypeScript | Go | C# | C++ | Ruby

### Tier 2: Great

Strong support; edge cases may need more guidance.

Rust | Java | PHP | Swift | Kotlin | Scala

### Tier 3: Config & Infra

Great at boilerplate, markup, and declarations.

HTML/CSS | SQL | YAML | JSON | Dockerfile | Bash

## POLYGLOT SUPERPOWERS

### Language Translation

Translates code between languages while preserving algorithms and idioms.

```
Chat: rewrite this Python function in Rust
```

### Framework Switching

Helps port components across major frameworks quickly.

```
Chat: Convert this React component to Vue 3
```

### Complex Query Generation

Generates SQL or regex from plain-English data requirements.

```
Comment: Regex to match valid IPv4
```

// WRITING FUNCTIONS WITH COPILOT

# The Comment-Driven Development Pattern

## THE 3-STEP CDD WORKFLOW

**01 State Intent**

Write a clear, natural language comment describing exactly what the function should do.

**02 Provide Context**

Add details like expected parameters, return types, or specific algorithms to use.

**03 Evaluate & Accept**

Review the generated ghost text. If it matches your intent, press Tab to accept.

### WHY IT WORKS

Copilot treats your comments as direct prompts. The more specific and structured your comment, the more accurate and production-ready the generated code will be.

## CDD IN ACTION

dataProcessor.js

```
1  // Function to calculate the Fibonacci sequence
2  // up to n numbers.
3  // Returns an array of integers.
4    function calculateFibonacci(n) {
5  if (n <= 0) return [];
6  if (n === 1) return [0];
7  let fib = [0, 1];
8  for (let i = 2; i < n; i++) {
9  fib.push(fib[i - 1] + fib[i - 2]);
10  }
11  return fib;
12  }
```

// REAL-WORLD EXAMPLE

# From Comment to Working API Endpoint in Minutes

## THE PROMPT (COMMENT)

routes.js

```
// Create an Express POST route for '/api/users'\n// Accept 'name' and 'email' and validate both exist\n// Save (mock) and return 201 with user
```

**Why this is a great prompt:**

- **Clear Intent:** Specifies framework (Express) and method (POST).
- **Defined Inputs:** Names expected payload fields (name, email).
- **Error Handling:** Dictates validation logic and status code (400).

## THE GENERATED CODE

routes.js

```
app.post('/api/users', (req, res) => {\n  const {name,email} = req.body;\n  if (!name || !email) return res.status(400).json({ error: 'Name and email are required' });\n  const newUser = { id: Date.now(), name, email };\n  users.push(newUser);\n  return res.status(201).json(newUser);\n});
```

// PROMPT ENGINEERING

# Getting Better Output Through Prompt Engineering

## THE 4 PILLARS OF PROMPTING

### 1. BE SPECIFIC

State exactly what you want. Avoid vague terms like "handle data" and use precise verbs like "parse JSON and extract user IDs".

### 2. PROVIDE CONTEXT

Mention the frameworks, libraries, or design patterns you want Copilot to utilize (e.g., "using Axios" or "as a React Hook").

### 3. GIVE EXAMPLES

Show the expected input and output formats directly in the comments. This drastically reduces hallucinated logic.

### 4. SET CONSTRAINTS

Explicitly state what Copilot should **not** do (e.g., "Do not use regex", "Do not mutate the original array").

## BEFORE & AFTER

**Vague Prompt**

```
// Parse the date
function parseDate(dateStr) {
```

Result: Copilot guesses the format, likely uses deprecated libraries, or writes fragile regex.

**Engineered Prompt**

```
// Parse ISO 8601 date string to a localized format (en-US).

// Input: "2023-10-25T10:00:00Z"
// Output: "10/25/2023"
// Constraint: Use native Intl.DateTimeFormat, do not use
moment.js
function parseDate(dateStr) {
```

Result: Copilot generates exact, robust, dependency-free code matching your architecture.

# Refactoring Legacy Code with Copilot

**BEFORE: Legacy Callback Hell**

```
function getUserData(userId, callback) {
db.query('SELECT * FROM users WHERE id=' + userId,
function(err, user) {
if (err) {
callback(err);
} else {
db.query('SELECT * FROM posts WHERE uid=' + user.id,
function(err2, posts) {
if (err2) {
callback(err2);
} else {
user.posts = posts;
callback(null, user);
}
});
}
});
}
```

**AFTER: Modern Async/Await**

```
// Refactored to use async/await and parameterized queries
async function getUserData(userId) {
try {
const user = await db.query('SELECT * FROM users WHERE id=?',
[userId]);
if (!user) throw new Error('User not found');

const posts = await db.query('SELECT * FROM posts WHERE
uid=?', [user.id]);
return { ...user, posts };

} catch (error) {
console.error('Error fetching user data:', error);
throw error;
}
}
```

**Prompt used:** "Refactor this function to use modern ES6 async/await instead of callbacks. Also, fix the SQL injection vulnerability by using parameterized queries."

// COPILOT MASTERY

# Generating Boilerplate and Scaffolding

**The Prompt (Top of File)**

```
// Express server with CORS and body-parser

// Define port 3000
```

server.js

```
const express = require('express');
const cors = require('cors');
const bodyParser = require('body-parser');

const app = express();
const PORT = process.env.PORT || 3000;

app.use(cors());
app.use(bodyParser.json());

app.get('/api/health', (req, res) => {
res.status(200).json({ status: 'UP', timestamp: new Date() });
});

app.listen(PORT, () => {
console.log(`Server is running on http://localhost:${PORT}`);
});
```

Press Tab to accept

# Translating Code Between Languages

**Legacy Python**

```python
def calculate_factorial(n):
if n < 0:
return None
if n == 0 or n == 1:
return 1

result = 1
for i in range(2, n + 1):
result *= i

return result
```

**Modern JavaScript**

```javascript
// Copilot translates the logic AND the idioms
function calculateFactorial(n) {
if (n < 0) {
return null;
}
if (n === 0 || n === 1) {
return 1;
}

let result = 1;
for (let i = 2; i <= n; i++) {
result *= i;
}

return result;
}
```

**Prompt used:** "Translate this Python function into modern JavaScript. Use camelCase naming conventions."

// COPILOT MASTERY

# Demystifying Regular Expressions

## 1. Generating Complex Regex

```
// Match a US Phone Number format: (123) 456-7890 or 123-456-7890
const                /^\(?([0-9]{3})\)?[-. ]?([0-9]{3})[-. ]?([0-9]{4})$/
```

## 2. Explaining Cryptic Regex

`/explain` What does /^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$/ do?

This is a standard **Email Validation** regular expression:

- ^[a-zA-Z0-9._%+-]+ : Matches the username part (letters, numbers, and basic symbols) before the @.
- @ Matches the literal "@" symbol.
- [a-zA-Z0-9.-]+ : Matches the domain name (like "gmail" or "company-name").
- \.[a-zA-Z]{2,}$ : Matches the top-level domain (like ".com" or ".io") ensuring it is at least 2 letters long at the end of the string.

// COPILOT MASTERY

# Automating Documentation (JSDoc/Docstrings)

Writing documentation is tedious but necessary. Type the comment trigger ( `/**` in JS/TS, or `"""` in Python) and Copilot generates typed docs from the function below.

api-service.js

```
/**
* Fetches user data from the API.
* @param {string} userId - The user ID.
* @returns {Promise<Object>} User data.
*/

async function fetchUserData(userId, includeHistory = false) {
if(!userId) throw new Error("User ID required");
const url = `/api/users/${userId}?history=${includeHistory}`;
const response = await fetch(url);
if(!response.ok) throw new Error("Network error");
return await response.json();
}
```

// COPILOT MASTERY

# Generating Realistic Mock Data

types.ts

```ts
interface Employee {
id: string;
fullName: string;
department: string;
hireDate: string;
isActive: boolean;
}
```

"Generate 4 realistic Employee objects across Engineering, HR, and Sales; make one inactive."

mockData.ts

```ts
export const mockEmployees: Employee[] = [
{
id: "EMP-8472",
fullName: "Sarah Jenkins",
department: "Engineering",
hireDate: "2021-03-15",
isActive: true
},
{
id: "EMP-9102",
fullName: "Marcus Chen",
department: "Sales",
hireDate: "2022-11-01",
isActive: true
},
{
id: "EMP-3391",
fullName: "Elena Rodriguez",
department: "HR",
hireDate: "2019-08-22",
isActive: false
},
{
id: "EMP-5514",
fullName: "James Wilson",
department: "Engineering",
hireDate: "2023-01-10",
isActive: true
}
];
```

Press Tab to accept

// COPILOT MASTERY

# Exploring Alternative Solutions

utils.js

```
// Works, but O(N^2)
function findCommonElements(arr1, arr2) { ... }
```

Copilot Chat

Too slow for large arrays. Any faster approach?

Use a Set for O(N) lookups. Optimized version:

```
function findCommonElements(a,b) { const s=new
Set(a); return [...new Set(b)].filter(x=>s.has(x)); }
```

// COPILOT MASTERY

# Handling Hallucinations & Bad Advice

## The Hallucination Problem

```
import { S3Client }   from "@aws-sdk/client-s3" ;

const client =   new S3Client ({ region:  "us-east-1"
});

// Copilot suggests:
const url =   client. getPublicUrl ("my-bucket" ,
"image.png" );
```

This method does not exist in AWS SDK v3!

Copilot is a probabilistic engine. It will sometimes invent methods, classes, or APIs that look completely plausible but do not actually exist in the library you are using.

## How to Protect Yourself

### Trust, but Verify

Never blindly accept code calling unfamiliar APIs. Hover over the generated method in VS Code to ensure IntelliSense recognizes it.

### Run Your Tests

Hallucinations compile perfectly in your head but fail at runtime. Rely on your unit tests and compiler errors to catch fake methods immediately.

### Feed it Documentation

If Copilot hallucinates, paste a snippet of the official documentation into your file or chat to ground the LLM in reality.

# Connectivity & Offline Scenarios



## Copilot is Cloud-Only

GitHub Copilot requires a constant, active internet connection to function. It does not run any LLM models locally on your machine. If you go offline, Copilot simply stops suggesting code.

## Connection Troubleshooting

1. **Corporate VPNs & Proxies**

   The #1 reason Copilot fails in enterprise environments is a corporate firewall blocking the telemetry or API endpoints. You may need to whitelist `*.githubcopilot.com` .

2. **Check the Output Log**

   If the icon is spinning but nothing happens, open the VS Code Output panel and select **GitHub Copilot** from the dropdown to view real-time network errors.

3. **Sign Out and Sign Back In**

   Sometimes the GitHub authentication token expires silently. Click the account icon in VS Code, sign out of GitHub, and sign back in to refresh the token.

// COPILOT MASTERY

# The Real ROI of GitHub Copilot

Copilot is not designed to replace developers; it is designed to keep them in the "flow state." Based on GitHub's internal research and industry case studies, the return on investment comes from reducing friction, not just writing code faster.



## 46%

### Code Written by AI

On average, nearly half of the code in files where Copilot is active is generated by the AI, drastically reducing keystrokes.



## 55%

### Faster Task Completion

Developers complete standard tasks (like writing an HTTP server or parsing data) significantly faster when assisted by Copilot.



## 75%

### Increased Satisfaction

Developers report feeling more fulfilled and less frustrated because they spend less time searching for boilerplate and syntax syntax.

// ADVANCED TECHNIQUES

# Copilot Chat Is a Full Development Partner

## BEYOND SIMPLE Q&A

### Iterative Refinement

Chat remembers the conversation history. Ask it to optimize, refactor, or add error handling to the code it just generated.

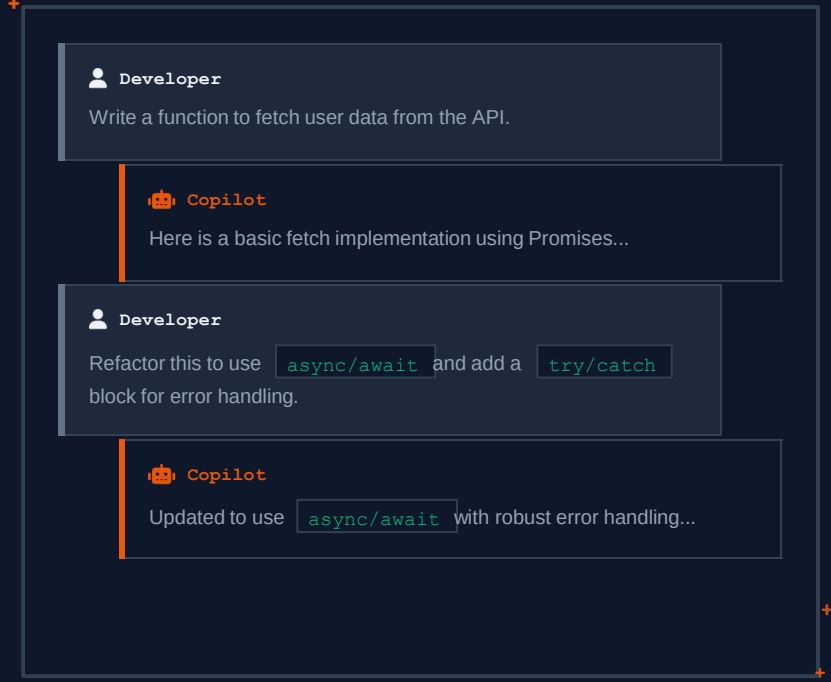
### Terminal Integration

When a build fails, select the terminal output and ask Chat to diagnose the error and suggest a fix.

### Code Review

Highlight your code and ask: "Are there any security vulnerabilities or performance bottlenecks here?"

## THE CONVERSATIONAL WORKFLOW



// SECURITY & BEST PRACTICES

# Using AI Assistance Responsibly and Securely

## CORE SECURITY PRINCIPLES

### Trust, but Verify

Copilot is an assistant, not an auditor. You are responsible for the code. Always review generated code and run standard SAST/DAST security tools.

### Protect Your Secrets

Never hardcode API keys, passwords, or tokens in comments to "help" Copilot. Use environment variables. Copilot reads your active files.

### Understand Data Privacy

Copilot Business and Enterprise tiers do **not** retain your code snippets or use your private repositories to train the public model.

## THE GOLDEN RULE

### VULNERABILITY PROPAGATION

### AI is a Reflection of its Context

Copilot adapts to the style and patterns of the surrounding code. **If your existing code contains security flaws, Copilot will likely generate more flaws to match your style.**

```
// If your file already has this:
db.query(`SELECT * FROM users WHERE id =
${req.body.id}`);
// Copilot will likely suggest this for the next
query:
db.query(`DELETE FROM posts WHERE id =
${req.body.postId}`);
```

Always establish secure patterns (like parameterized queries) early in your files so Copilot learns the correct, secure way to assist you.

// DAY 1 RECAP

# You Have Built the Foundation — Here Is What You Now Know



## GIT & GITHUB

- **Version Control Concepts**
  Understanding repositories, commits, and history.
- **The 3 Stages of Git**
  Working Directory, Staging Area, and Local Repo.
- **Branching & Merging**
  Isolating features and resolving conflicts.



## COPILOT SETUP

- **The LLM Engine**
  How Copilot predicts tokens instead of searching.
- **Installation & Interfaces**
  Setting up the extension in VS Code.
- **Context Windows**
  How open tabs and files influence suggestions.



## COPILOT MASTERY

- **Comment-Driven Dev**
  Using natural language to generate functions.
- **Prompt Engineering**
  Being specific, contextual, and setting constraints.
- **Iterative Chat**
  Refining, refactoring, and explaining code.

> SYSTEM_STATUS: READY_FOR_DAY_2

// LOOKING AHEAD

# Tomorrow: Automation, CI/CD, and Project-Based Collaboration

## DAY 2 AGENDA

**01 GitHub Actions Fundamentals**

Understanding workflows, events, runners, jobs, and steps.

**02 Building CI/CD Pipelines**

Automating testing, linting, and deployment processes directly from your repository.

**03 Copilot + Actions**

Using AI to write, debug, and optimize complex YAML workflow files.

**04 Team Collaboration**

Branch protection rules, code owners, and pull request review workflows.

## PREPARATION & HOMEWORK

**Complete Day 1 Labs**

Ensure all Git fundamentals and Copilot configuration exercises are finished.

**Verify Actions Access**

Check your GitHub repository settings to ensure GitHub Actions is enabled for your account.

**Review YAML Syntax**

Brush up on basic YAML formatting, as it is the foundation for all GitHub Actions workflows.

// RESOURCES & REFERENCES

# Keep Learning After the Bootcamp

## OFFICIAL DOCS

**docs.github.com**
The primary source of truth for all GitHub features, including Actions and Copilot.

**GitHub Blog**
Stay updated on the latest Copilot model upgrades and new feature releases.

## COMMUNITY

**GitHub Community**
Forums for asking questions, sharing workflows, and discussing best practices.

**Awesome Actions**
A curated list of awesome things related to GitHub Actions (found on GitHub).

## PRACTICE LABS

**GitHub Skills**
Interactive courses built into GitHub repositories to learn by doing.

**Microsoft Learn**
Free, structured learning paths for GitHub Copilot and CI/CD pipelines.

### End of Day 1 Materials

Review your notes, complete the optional labs, and prepare for Day 2.

PREPARE FOR DAY 2

// BOOTCAMP · 2026 · DAY 2 OF 2

# GitHub Actions + CI/CD Automation

Day 2: Advanced Workflows, Automated Pipelines, and Enterprise Security

</> 10:00 AM – 5:00 PM



Advanced Collaboration Workflows

GitHub Actions Fundamentals

CI/CD Pipelines in Practice

Real-World Project Build

Security & Licensing

AUDIENCE: Developers & GitHub Admins

FORMAT: Lecture + Hands-On Labs

LABS: 10 Hands-On Exercises

// TODAY'S ROADMAP

# Day 2 Agenda

| | | |
|---|---|---|
| 10:00 – 11:15 | Advanced GitHub Workflows & Collaboration | Branch protection, CODEOWNERS, Issues, Projects |
| 11:30 – 1:00 | GitHub Actions Deep Dive | Triggers, runners, secrets, caching, reusable workflows, composite actions |
| 2:00 – 3:00 | CI/CD Pipelines & Cloud Deployment | Testing, matrix builds, Azure/AWS deployment, environments |
| 3:15 – 4:15 | Copilot + Actions: Building a Real Project | Using Copilot to scaffold code and generate CI/CD workflows end-to-end |
| 4:15 – 5:00 | Security, Licensing & Wrap-Up | GHAS, CodeQL, Dependabot, AI code licensing, admin controls |

// LEARNING OBJECTIVES

# By End of Day, You Will Be Able To…

Six concrete automation and security skills you will practice and demonstrate through hands-on labs today.

Developers GitHub Admins Team Leads

1. Configure branch protection and CODEOWNERS
2. Author GitHub Actions workflows from scratch
3. Build a complete CI/CD pipeline with tests and deploy
4. Use Copilot to scaffold and automate a real project
5. Apply GHAS, CodeQL, and Dependabot for security
6. Manage AI licensing risk and admin-level controls

// DAY 1 RECAP

# The Foundation We Built Yesterday

### Git & GitHub Fundamentals

- Working directory, staging, commits, branches
- Pull requests & code review workflow
- Merge conflict resolution
- 7 core commands used daily

### GitHub Copilot Mastery

- Installing and configuring Copilot in VS Code
- Inline suggestions, chat, and slash commands
- Prompt engineering for accurate results
- Security and responsible AI use

### Today We Go Further

Yesterday you learned to write and review code with Copilot's help.

Today, we automate everything around that code: enforcing quality gates, running tests on every push, and deploying safely to production.

Same tools, same repo – now wired together into a real CI/CD pipeline.

// SECTION 1: ADVANCED WORKFLOWS

# Protecting main Is a Team Sport, Not a Solo Habit

### Without Guardrails

- Anyone can push straight to main
- Broken code reaches production unnoticed
- No required reviewer before merge
- Force-pushes silently rewrite history

### With Branch Protection

- Required status checks before merge
- Mandatory reviewer approval via CODEOWNERS
- Linear history, no accidental force-pushes
- Automated CI must pass before merge

// BRANCH PROTECTION IN PRACTICE

# Settings → Branches → Add Rule

1

### Require pull request reviews before merging

No one, including admins, can merge without at least one approval.

2

### Require status checks to pass before merging

CI must be green: lint, tests, and build all report success.

3

### Restrict who can push to matching branches

Only release managers or bots can push directly to main.

**Live Demo:** Try pushing directly to main with these rules on – watch GitHub reject it.

// AUTOMATIC REVIEW ASSIGNMENT

# CODEOWNERS: The Right Reviewer, Every Time

.github/CODEOWNERS

```
*.js   @frontend-team
*.py   @backend-team
/docs/   @tech-writers
/infra/   @platform-team
```

Syntax: path/to/files @team-or-user

## Real-World Impact

At a company with 50 developers, no single reviewer can vet every change with confidence.

CODEOWNERS automatically requests the right expert the moment a matching file changes – frontend engineers see JS changes, backend engineers see Python changes, writers see docs.

Combined with required reviews, this guarantees domain expertise on every single merge.

// THE UNIT OF WORK

# GitHub Issues: Where All Work Begins

### Bug

Something is broken and needs a fix

### Feature Request

New capability to be designed and built

### Task

A discrete unit of engineering or ops work

### Question

Clarification needed before work starts

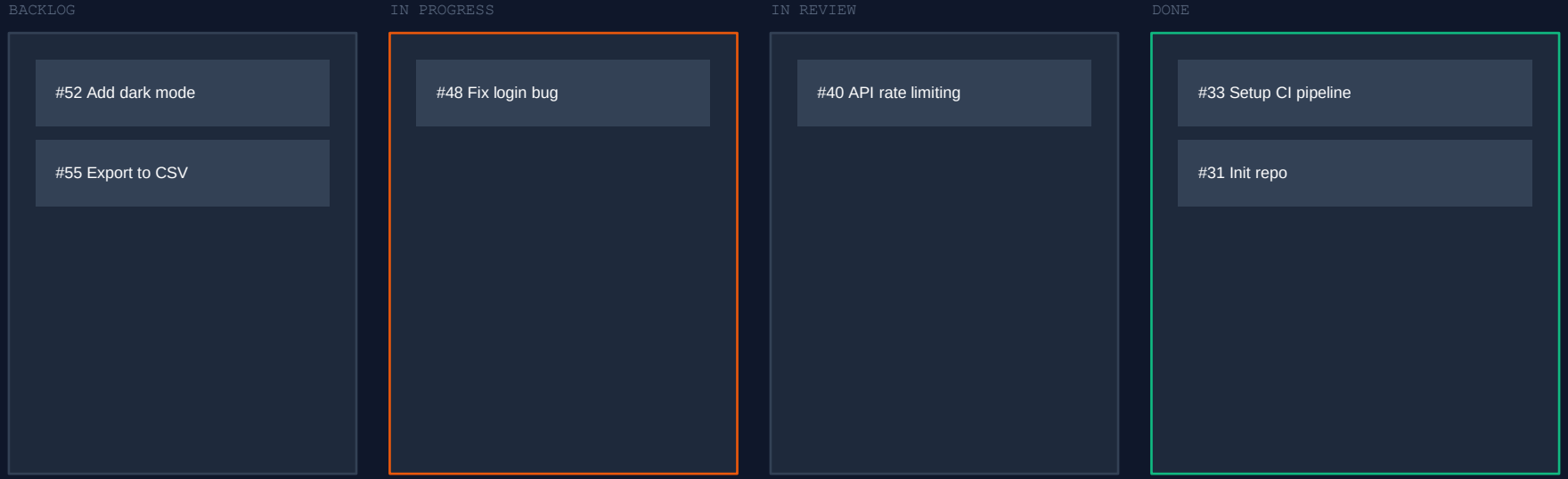
LABELS & MILESTONES

`priority:high` `type:bug` `status:in-progress`

Full traceability: an **Issue** links to a **PR**, which links to a **commit**, which triggers a **CI run**.

// KANBAN FOR DEVELOPERS

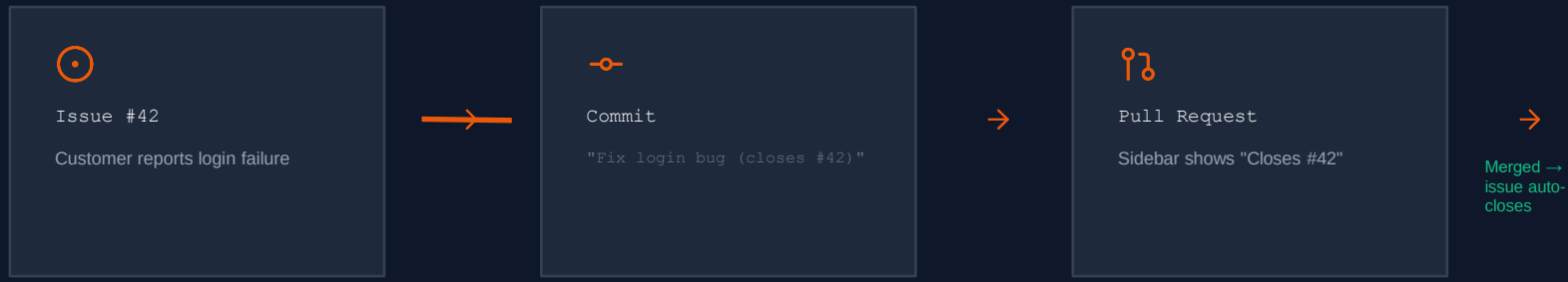
# GitHub Projects: Your Team's Board




Custom Fields: Status · Priority · Sprint · Estimate  Automation: auto-move cards on PR merge

// FULL TRACEABILITY

# Linking Issues, PRs, and Commits




Real-World: this creates a full audit trail from a customer bug report to the exact line of code that fixed it – invaluable during incident reviews and compliance audits.

// REVIEWING WITH PURPOSE

# Code Review Best Practices

- Review the "why," not just the "what" – question the approach, not just the syntax
- Use "Suggest Changes" for small fixes instead of comment threads
- Approve only when you would be comfortable owning this code in production
- Use Copilot as a second reviewer before requesting a human review

COPILOT PROMPT

```
"Does this code have any security issues?"
```

// SECTION 2: GITHUB ACTIONS FUNDAMENTALS

# From Manual to Automated

Every push, pull request, and release can now trigger a workflow that lives inside your repository.

// GITHUB ACTIONS

# What Is GitHub Actions?

## A CI/CD Platform Built Into GitHub

Workflows are defined as YAML files in .github/workflows/

Triggered by events: push, pull_request, schedule, or a manual dispatch

Runs on GitHub-hosted or self-hosted runners

No external CI service to configure – it lives in the same repo as your code

### GitHub Actions

Native automation for build, test, and deploy – no Jenkins or CircleCI required.

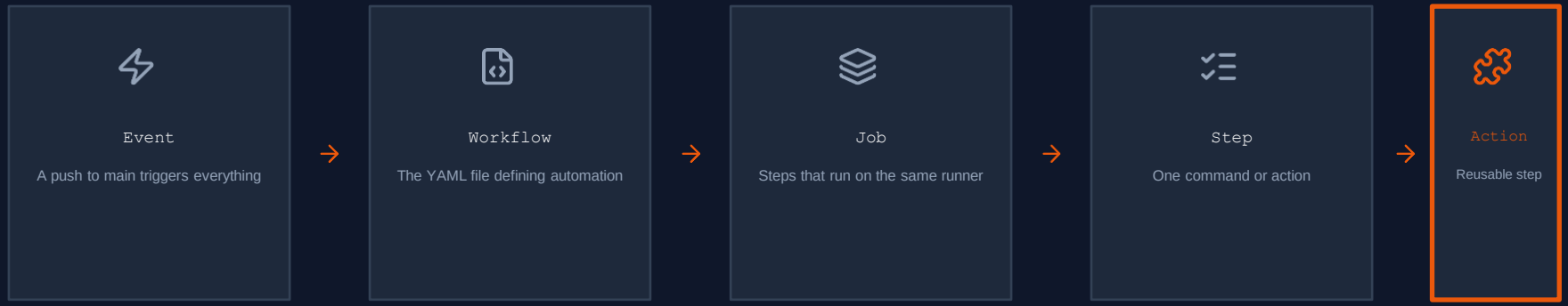
### Runners

Ephemeral virtual machines that execute your workflow steps.

**Free** 2,000 MINUTES/MONTH FOR PRIVATE REPOS

// ACTIONS ARCHITECTURE

# Event → Workflow → Job → Step → Action




i An Action is a reusable, pre-built Step pulled from the Marketplace.

// WORKFLOW SYNTAX

# Anatomy of a Workflow YAML File

.github/workflows/ci.yml

```yaml
name: CI

on: [push, pull_request]

jobs:

  build:

    runs-on: ubuntu-latest

    steps:

      - uses: actions/checkout@v4

      - run: npm test
```

**name**

Human-readable name for the workflow

**on**

The event(s) that trigger the workflow

**jobs / runs-on**

One or more jobs, each with a target OS

**steps**

The ordered list of commands/actions in the job

// YOUR FIRST WORKFLOW

# Hello World, From a Runner

.github/workflows/hello.yml

```yaml
name: Hello World
on: push
jobs:
  greet:
    runs-on: ubuntu-latest
    steps:
```

STEP 1

`uses: actions/checkout@v4`

Checks out the repository code onto the runner

STEP 2

`run: echo "Hello from Actions!"`

Executes a shell command on the runner

// GITHUB-HOSTED RUNNERS

# Choosing the Right Runner OS

| RUNNER | BEST FOR | SPECS (FREE TIER) |
| --- | --- | --- |
| ubuntu-latest | Most CI/CD tasks, fastest startup | 2-core, 7GB RAM, 14GB SSD |
| windows-latest | .NET, PowerShell workflows | 2-core, 7GB RAM, 14GB SSD |
| macos-latest | iOS / macOS app builds | 3-core, 14GB RAM, 14GB SSD |

**PRO TIP:** For most CI/CD tasks, ubuntu-latest is the cheapest and fastest choice.

// SELF-HOSTED RUNNERS

# When GitHub-Hosted Isn't Enough

### Why

Larger compute, access to private networks, and custom pre-installed software.

### Setup

Download the runner agent, register it with a token, then run it as a service.

### Use Cases

GPU workloads, database integration tests, air-gapped environments.

### Security Warning

Never use self-hosted runners on public repositories – any contributor's PR can execute arbitrary code on your infrastructure.

// DON'T REINVENT THE WHEEL

# The GitHub Actions Marketplace

`actions/checkout@v4`

Pulls your repository code onto the runner – used in almost every workflow.

`actions/setup-node@v4`

Installs a specific Node.js version before running install/test steps.

`actions/upload-artifact@v4`

Saves build output or test reports for later download or deployment steps.

Security Tip: pin third-party actions to a full commit SHA, not just a version tag, to avoid supply-chain attacks.

// TRIGGERS DEEP DIVE

# Five Ways to Kick Off a Workflow

| TRIGGER | WHEN IT FIRES |
| --- | --- |
| `push` | On every push, filterable by branch or path |
| `pull_request` | On PR open, sync (new commits), or reopen |
| `schedule` | Cron syntax – runs on a fixed timer |
| `workflow_dispatch` | Manual trigger with optional inputs |
| `release` | Fires when a GitHub Release is published |

// ENVIRONMENT VARIABLES & SECRETS

# Never Hardcode a Secret

### env

Define variables at workflow, job, or step level

```
env: NODE_ENV: production
```

### secrets

Encrypted values stored in GitHub Settings

```
${{ secrets.MY_API_KEY }}
```

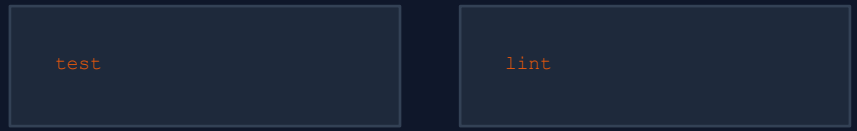
### Security Rule

Never hardcode secrets in YAML – they remain permanently visible in git history. GitHub automatically masks secret values in workflow logs.

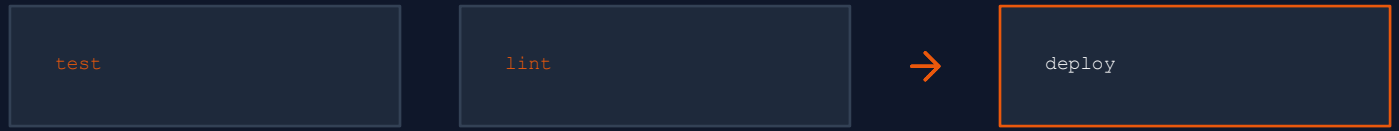
// JOB DEPENDENCIES

# Parallel by Default, Sequential by Choice

DEFAULT: RUNS IN PARALLEL




WITH needs: [test, lint]




Real-World: a 10-minute pipeline becomes 5 minutes by running test and lint in parallel, then triggering deploy only once both succeed.

// CACHING FOR SPEED

# actions/cache: Skip the Rework

CACHE KEY

```
key: ${{ hashFiles('package-lock.json') }}
```

A hash of your lock file – cache node_modules to skip npm install on every run.

IMPACT

4 min → 45 sec

Typical pipeline time reduction after caching node_modules

```
- uses: actions/cache@v4
  with:
    path: node_modules
    key: ${{ runner.os }}-${{ hashFiles('**/package-lock.json') }}
```

// SHARING DATA BETWEEN JOBS

# Artifacts: Build Once, Use Everywhere

Job: build

actions/upload-artifact@v4

Save files from a job – test reports, build outputs, binaries



Job: deploy

actions/download-artifact@v4

Load the same files in a later job in the pipeline

Real-World: build the binary once in the build job, then test that exact same binary across multiple environments – no re-compilation, no drift.

// SECTION 3: CI/CD IN PRACTICE

# What Is CI/CD?

### CI

Continuous Integration

Automatically build and test every code change the moment it's pushed.

### CD

Continuous Delivery

Automatically prepare every change so it's always ready to release.

### CD

Continuous Deployment

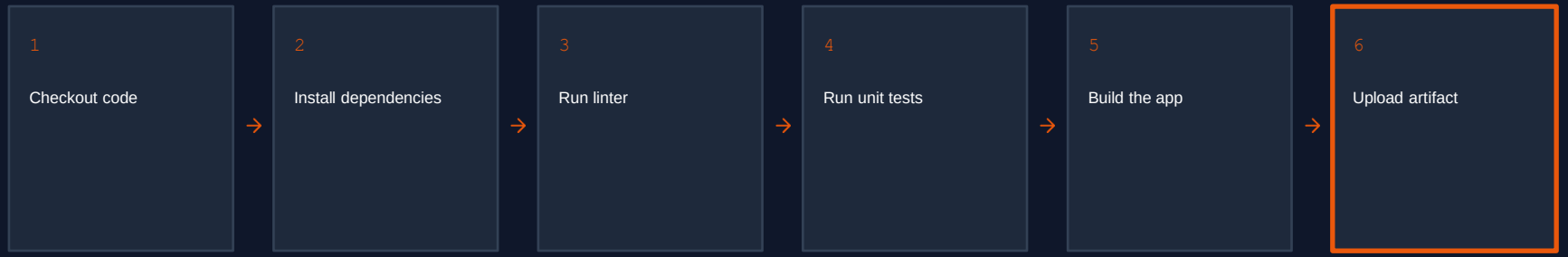
Automatically deploy every passing change straight to production.

THE GOAL

Reduce the time from "code written" to "code in production" from weeks to minutes.

// THE STANDARD CI PIPELINE

# Six Stages, Every Time




Every green checkmark on this path means the code is one step closer to production.

// BUILDING A NODE.JS CI PIPELINE

# The Most Common Pipeline in the World

.github/workflows/node-ci.yml

```yaml
on: push
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 20 }
      - run: npm ci
      - run: npm run lint
      - run: npm test
```

← faster & reliable vs npm install

// TEST REPORTING

# See Failures, Not Just Logs

PR CHECKS SUMMARY

- test_login_success – passed in 42ms
- test_login_invalid_password – failed: expected 401, received 500
- test_create_task – passed in 18ms
- test_delete_task – passed in 22ms

Tools: actions/upload-artifact for JUnit XML · dorny/test-reporter@v1 for rich annotations

// MATRIX BUILDS

# Test Every Combination at Once

`strategy.matrix: node [18, 20, 22] × os [ubuntu, windows, macos]`

| | | |
|---|---|---|
| ✓ Node 18 · ubuntu-latest | ✓ Node 18 · windows-latest | ✓ Node 18 · macos-latest |
| ✓ Node 20 · ubuntu-latest | ✓ Node 20 · windows-latest | ✓ Node 20 · macos-latest |
| ✗ Node 22 · ubuntu-latest | ✓ Node 22 · windows-latest | ✓ Node 22 · macos-latest |

Impact: catch cross-platform bugs – like the failing Node 22 build above – before they reach production.

// CODE QUALITY GATES

# Automating Code Quality with Linters

### ESLint

JavaScript / TypeScript

Catches unused variables, unsafe patterns, and style violations.

### Flake8 / Black

Python

Enforces PEP8 style and auto-formats code consistently.

### Prettier

Formatting, any language

Opinionated formatter that removes debates over whitespace and quotes.

Real-World: a linter in CI ensures every developer on a 50-person team writes code in the same style, eliminating style debates in code reviews.

// DON'T REPEAT YOURSELF

# Reusable Workflows

shared-repo/.github/workflows/ci.yml

```yaml
on:
  workflow_call:
    inputs:
      node-version: {type: string}

jobs:
  build:
    steps: [checkout, install, lint, test]
```

team-repo-a/.github/workflows/ci.yml

```yaml
jobs:
  call-shared-ci:
    uses: org/shared-repo/.github/
      workflows/ci.yml@main
    with:
      node-version: "20"
```

Real-World: an org with 80 repos maintains ONE canonical CI workflow instead of copy-pasting YAML 80 times.

// PACKAGING YOUR OWN STEPS

# Composite Actions

.github/actions/setup-node-env/action.yml

```yaml
runs:
  using: "composite"
  steps:
    - uses: actions/setup-node@v4
```

BEFORE (repeated in every workflow)

```
- uses: actions/setup-node@v4
- run: npm ci
- run: npm run build
```

AFTER (one reusable step)

```
- uses: ./.github/actions/setup-node-env
```

Real-World: a platform team ships a composite action once; every product team's workflow shrinks from 15 lines to 1.

// DEPLOYING TO THE CLOUD

# Deploying to Azure App Service

## Setup Checklist

Action: azure/webapps-deploy@v3

Secret needed: AZURE_WEBAPP_PUBLISH_PROFILE

Trigger: only on push to main branch

Pattern: build in CI job → deploy in CD job, linked with needs:

deploy job

```yaml
deploy:
  needs: build
  if: github.ref == 'refs/heads/main'
  steps:
    - uses: azure/webapps-deploy@v3
      with:
        publish-profile: ${{ secrets.AZURE... }}
```

// DEPLOYING STATIC SITES

# Deploying to AWS: S3 + CloudFront

AUTH: aws-actions/configure-aws-credentials@v4

1

**Build the App**

`npm run build`

2

**Sync to S3**

`aws s3 sync ./build s3://my-bucket`

3

**Invalidate CDN Cache**

`CloudFront create-invalidation`

Real-World: this exact three-step pattern is how most static websites (React, Vue, Angular) are deployed to production.

// ENVIRONMENT PROTECTION

# Environment-Specific Deployments

## staging

Deploys automatically on every merge to main – no approval needed.

SECRETS

STAGING_API_KEY

## production

Protection rule: requires manual approval from a designated reviewer before deploying.

SECRETS

PRODUCTION_API_KEY

Real-World: a change goes to staging automatically, but requires a manager's approval before going to production.

// ALERTING ON FAILURE

# Notifications and Alerting

#deployments – Slack

⚠ Production deploy FAILED – workflow run #482

Triggered by: jsmith · View run →

TOOLS

slackapi/slack-github-action@v1

dawidd6/action-send-mail@v3

TRIGGER CONDITION

```
if: failure()
```

Real-World: the on-call engineer gets a Slack message the moment a production deployment fails, with a link to the failing run.

// WHEN WORKFLOWS FAIL

# Workflow Debugging Techniques

`ACTIONS_STEP_DEBUG = true`

Set as a repo secret to enable verbose step-by-step logging

`mxschmitt/action-tmate@v3`

Opens an SSH session directly into a live, failing runner

```
echo "Value is: ${{ steps.build.outputs.x }}"
```

Print variable values directly into the log output

Expand each step in the GitHub Actions UI to see full output

Always the first place to look before adding debug tooling

// SECTION 4: PROJECT-BASED COLLABORATION

# Putting It All Together

Git, Copilot, and Actions – now we build a real mini-application using everything we've learned.

// THE MINI-APP

# Project Architecture: Task Manager API

## Node.js / Express REST API

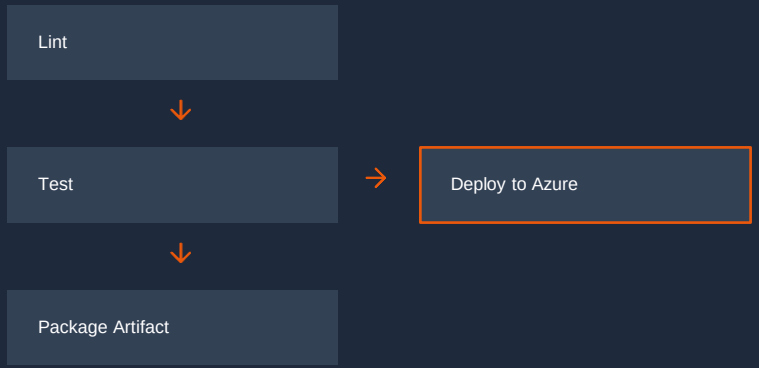
ENDPOINTS

```
GET    /tasks
POST   /tasks
DELETE /tasks/:id
```

TESTS

Jest unit tests for every endpoint

## Pipeline



// LIVE DEMO

# Using Copilot to Scaffold the Project

1. Create a new repo on GitHub
2. Open VS Code, use Copilot Chat to generate the Express server
   `"Create an Express server with GET/POST/DELETE /tasks endpoints"`
3. Use Copilot to generate the Jest test file
4. Use Copilot to generate the GitHub Actions CI workflow file
   `"Create a workflow that lints and runs the Jest tests on every PR"`

Live Demo: watch Copilot generate 80% of the boilerplate in under 5 minutes.

// THE META MOMENT

# Using Copilot to Write the CI Workflow

COPILOT CHAT PROMPT

"Write a GitHub Actions workflow that lints, tests, and deploys this Node.js app to Azure on merge to main"



GENERATED: .github/workflows/ci.yml

```
jobs:
  build:
    steps: [lint, test, deploy-to-azure]
```

Review this generated YAML carefully before merging – Copilot writes fast, but you own correctness.

// REAL-TIME COLLABORATION

# Resolving Merge Conflicts

Scenario: two developers modify the same file simultaneously.

1. Developer A merges their PR first
2. Developer B's PR now shows a merge conflict
3. Resolve in VS Code using the 3-way merge editor



COPILOT TIP

Highlight the conflict markers and use /explain to understand both changes before deciding.

// SNAPSHOTTING YOUR CODE

# Managing Releases with GitHub Releases

## What Is a Release?

A snapshot of the repository at a specific point in time.

Tied to a Git tag, e.g. v1.0.0, v2.3.1

Includes release notes and attached binaries such as build artifacts.

Automation trigger: on: release: types: [published]

RELEASE v2.3.1

Task Manager API v2.3.1

- Fixed pagination bug - Added rate limiting

task-api-v2.3.1.tar.gz

Attached as a build artifact

// VERSIONING WITH MEANING

# Semantic Versioning (SemVer)

2.3.1

**MAJOR**

`v1.0.0 → v2.0.0`

Breaking changes to the API contract

**MINOR**

`v1.0.0 → v1.1.0`

New, backward-compatible features

**PATCH**

`v1.0.0 → v1.0.1`

Backward-compatible bug fixes

Real-World: every npm package and GitHub Release should follow SemVer so consumers know what to expect.

// AUTOMATED DEPENDENCY UPDATES

# Dependabot: Fighting Dependency Rot

.github/dependabot.yml

```
version: 2
updates:
  - package-ecosystem: "npm"
    schedule: { interval: "weekly" }
```

Auto-generated PR #91

Bump express from 4.18.1 to 4.18.3

CI runs automatically – full test suite must pass before merge

This is how enterprise teams avoid the slow accumulation of outdated, vulnerable packages.

// SECTION 5: SECURITY AND LICENSING

# Security in GitHub Actions

**Principle of Least Privilege**

Only grant the permissions the workflow actually needs

permissions: BLOCK

```yaml
permissions:
  contents: read    # for a test workflow
  packages: write   # for a publish workflow
```

Secret Scanning: GitHub automatically scans every push for accidentally committed secrets.

// GITHUB ADVANCED SECURITY (GHAS)

# The Enterprise Security Layer



## Code Scanning

Uses CodeQL to find vulnerabilities in your code automatically.



## Secret Scanning

Detects API keys, tokens, and passwords in commits.



## Dependency Review

Blocks PRs that introduce vulnerable dependencies.



GHAS is free for public repos. Private repos require GitHub Enterprise.

// SEMANTIC CODE ANALYSIS

# CodeQL: Automated Code Scanning

WORKFLOW STEPS

```
- uses: github/codeql-action/init@v3
- uses: github/codeql-action/analyze@v3
```

SUPPORTED LANGUAGES

JavaScript | TypeScript | Python | Java | C# | Go | Ruby

Real-World: findings appear as annotations directly on the PR – a CodeQL scan once caught a SQL injection vulnerability before it merged to production.

// AI CODE LICENSING

# What You Need to Know

### The Risk

Copilot is trained on public code, which may include GPL-licensed code. It may suggest code substantially similar to a licensed work.

### The Mitigation

Enable "Duplication Detection" in Copilot settings to filter suggestions matching public code.

Enterprise Policy: many companies require all AI-generated code to be reviewed by a human before merge.

// MANAGING COPILOT AT SCALE

# Copilot for Business: Admin Controls



### Org-Level Policy

Enable or disable Copilot for specific teams



### Seat Management

Assign and revoke Copilot licenses from org settings



### Content Exclusions

Exclude /secrets/, /legal/ from Copilot suggestions



### Audit Logs

Track who is using Copilot and when

// DAY 2 RECAP

# The Full Picture

### Advanced Workflows
Branch protection, CODEOWNERS, Issues, Projects

### GitHub Actions
Triggers, jobs, runners, secrets, caching

### CI/CD
Lint, test, build, deploy pipelines

### Project Collaboration
Real-world app build with Copilot end-to-end

### Security
GHAS, CodeQL, licensing, admin controls

// CONTINUING YOUR JOURNEY

# What's Next

### Certifications

GitHub Foundations, Actions, Advanced Security

### Resources

docs.github.com · github.com/skills · GitHub Blog

### Community

GitHub Community Forum · GitHub Universe

### Practice

Build a personal project using everything from this bootcamp

Thank you for completing the bootcamp!